

Report on GraphRT — Reimplementation of DeepGCNRT by Kang et al.

Deyi Wang, Leyan Cheng, Jiani Xu, Yuxin Gong

April 15, 2026

Contents

1	Introduction and Paper Summary	2
2	Model Architecture Reimplementation	2
2.1	Graph Construction	2
2.2	Message Passing Layer	2
2.3	Readout and Prediction Head	3
2.4	Assumptions and Design Decisions	4
3	Implementation Details	4
4	Experimental Setup	5
4.1	Datasets and Preprocessing	5
4.2	Training Configuration	5
4.3	Deviations from the Original Experimental Protocol	6
5	Results and Comparison	6
5.1	SMRT Benchmark	6
5.2	MiniDataset Transfer Learning on the Seven Paper Datasets	7
5.3	Additional Datasets Present in the Official Code	7
6	Analysis and Discussion	8
6.1	Why Do Some Results Match Well While Others Do Not?	8
6.2	Sensitivity to Hyperparameters and Randomness	9
6.3	Strengths and Weaknesses of the Architecture	9
6.4	What We Learned from the Reimplementation	10
7	Conclusion and Lessons Learned	10

1 Introduction and Paper Summary

Retention time (RT) prediction is a core problem in LC-MS analysis because RT provides information that is complementary to MS/MS spectra. In the target paper, Kang et al. proposed **DeepGCN-RT**, a deep graph neural network for small-molecule RT prediction, and argued that previous RT-GNN models were still relatively shallow. Their main claim is that a **16-layer** graph convolutional architecture can be trained effectively once **residual connections**, **edge-aware message passing**, and an **attention/GRU readout** are introduced [1].

The paper reports three main contributions. First, on the SMRT benchmark, DeepGCN-RT improves the previously reported state of the art and reaches a test MAE of **26.55 s** and a MAPE of **0.033** (3.3%). Second, after transfer learning from SMRT, the model improves performance on seven external chromatographic datasets with different conditions. Third, the authors show that predicted RT can help molecular identification on the RIKEN-PlaSMA dataset by filtering candidate structures before final ranking.

Our project focuses on an **independent PyTorch Geometric reimplementation** of the paper’s core model and training pipeline. We used the authors’ official repository only to clarify underspecified choices such as the readout structure, output normalization, and the exact transfer-learning hyperparameters. The final code in this repository was written independently and integrated into a clean training pipeline centered on PyG datasets, PyG data loaders, and our own `Networks/DeepGCNRT.py`. The final experiments reported here focus on SMRT base-model training and direct SMRT-initialized transfer learning on each MiniDataset under a **single shuffled train/validation/test split**. We do not include our auxiliary RIKENMONA regression run in the main report because it does not match the paper’s full RIKEN-PlaSMA identification protocol.

2 Model Architecture Reimplementation

2.1 Graph Construction

Each molecule is converted from a canonical SMILES string into a graph $G = (V, E)$, where atoms are nodes and bonds are directed edges. The preprocessing and featurization code in this repository produces node features of dimension **164** and edge features of dimension **11**. This closely follows the feature design described in the paper and the earlier GCN work by Kensert et al., which the paper explicitly adopts [1].

The node representation contains 20 groups of atom-level features, including element type, degree, formal charge, hybridization, aromaticity, hydrogen-bond donor/acceptor flags, ring membership, chirality-related features, and several RDKit-derived physicochemical contributions. Edge features contain 5 groups of bond-level attributes, including bond type, stereo information, conjugation, ring membership, and rotatability.

2.2 Message Passing Layer

The reimplemented model follows the paper’s notation as closely as possible. Let $\mathbf{h}_u^{(l)} \in \mathbb{R}^d$ denote the hidden representation of source node u at layer l , and let $\mathbf{h}_{e_{uv}}^{(l)} \in \mathbb{R}^d$ denote the encoded edge feature between u and v . The edge-aware message is

$$\mathbf{m}_{uv}^{(l)} = \mathbf{h}_u^{(l)} + \mathbf{h}_{e_{uv}}^{(l)}. \quad (1)$$

Following the best-performing setting used by the authors’ released code, our implementation uses the `no_relu` aggregation variant, so the attention weight is computed directly from the message:

Algorithm 1 Forward Pass of Our DeepGCN-RT Reimplementation

```
1: Encode atom features with a linear layer:  $\mathbf{x}_v \mapsto \mathbf{h}_v^{(0)}$ 
2: Encode bond features with a linear layer:  $\mathbf{e}_{uv} \mapsto \mathbf{h}_{e_{uv}}^{(0)}$ 
3: for  $l = 0, \dots, 15$  do
4:   Compute edge-aware messages  $\mathbf{m}_{uv}^{(l)}$ 
5:   Aggregate incoming messages with softmax-normalized weights
6:   Apply linear layer, ReLU, dropout, and residual connection
7: end for
8: Initialize graph embedding by sum pooling over all nodes
9: for  $k = 1, 2$  do
10:  Update graph embedding with graph attention and a GRU cell
11: end for
12: Predict RT with the MLP head
```

$$\alpha_{uv}^{(l)} = \frac{\exp\left(\mathbf{m}_{uv}^{(l)}\right)}{\sum_{u' \in \mathcal{N}(v)} \exp\left(\mathbf{m}_{u'v}^{(l)}\right)}. \quad (2)$$

Messages are aggregated at the destination node by weighted summation:

$$\mathbf{m}_v^{(l)} = \sum_{u \in \mathcal{N}(v)} \alpha_{uv}^{(l)} \odot \mathbf{m}_{uv}^{(l)}. \quad (3)$$

The node state is then updated by a linear layer, ReLU, dropout, and residual connection:

$$\mathbf{h}_v^{(l+1)} = \text{Norm}\left(\text{Dropout}\left(\text{ReLU}\left(W^{(l)} \mathbf{m}_v^{(l)}\right)\right) + \mathbf{h}_v^{(l)}\right). \quad (4)$$

In our final configuration, Norm is the identity map, which matches the released DeepGCN-RT implementation for the edge-aware residual model.

2.3 Readout and Prediction Head

After $L = 16$ graph convolution layers, molecule-level embeddings are produced by an attentive readout with two refinement steps. The initial graph state is the sum of node embeddings:

$$\mathbf{g}^{(0)} = \sum_{v \in V} \mathbf{h}_v^{(L)}. \quad (5)$$

At each readout step, a graph-attention module propagates information from nodes to the graph state, and a GRU cell updates the graph embedding:

$$\mathbf{c}^{(k)} = \text{GAT}\left(\{\mathbf{h}_v^{(L)}\}, \mathbf{g}^{(k-1)}\right), \quad \mathbf{g}^{(k)} = \text{GRU}\left(\mathbf{c}^{(k)}, \mathbf{g}^{(k-1)}\right). \quad (6)$$

The final molecular representation is passed through a two-layer MLP:

$$\mathbb{R}^{200} \rightarrow \mathbb{R}^{1024} \rightarrow \mathbb{R}^1,$$

with a ReLU between the two linear layers. The scalar output is the predicted RT in seconds.

2.4 Assumptions and Design Decisions

Several details are not fully specified in the paper text, so we made the following explicit choices:

1. **Independent framework choice.** The authors’ code is written with DGL. We reimplemented the same layer structure in **PyTorch Geometric**, using `GATConv`, `global_add_pool`, and `GRUCell`. This changes the backend library and some internal operator details, even when the high-level computation graph is kept close to the paper.
2. **Output normalization.** The released DeepGCN-RT code uses `output_norm="none"` for the edge-aware attentive model. We follow that setting.
3. **Aggregation update function.** The released code exposes three options: `relu_eps_beta`, `relu`, and `no_relu`. We adopt `no_relu`, because it is the default path used by the published training scripts for DeepGCN-RT.
4. **Readout approximation.** The official DGL implementation directly uses `AttentiveFPReadout`. PyG does not provide a drop-in equivalent with the same implementation details, so we approximate the readout with graph attention plus `GRUCell` refinement. This is functionally close, but not guaranteed to be numerically identical.
5. **Readout initialization.** The paper states that the initial graph embedding is obtained by summing node embeddings before the GRU refinement. Our readout reproduces exactly that behavior.
6. **Loss implementation.** The paper reports Huber loss. In PyTorch, `SmoothL1Loss` is the standard Huber-style implementation, so we use it as the training objective.

3 Implementation Details

The codebase was reorganized around three layers:

1. **Dataset modules.** `Datasets/SMRT.py` and `Datasets/MiniDatasets.py` expose the shared interface used by the final reported experiments for preparing raw data, building PyG graphs, and loading train/validation/test splits.
2. **Model definition.** `Networks/DeepGCNRT.py` contains the atom encoder, bond encoder, 16 residual edge-aware GCN layers, attentive readout, and final prediction head.
3. **Training utilities.** `Train/lib/train_graph.py` implements the epoch loop, early stopping, scheduler stepping, checkpoint selection by validation loss, and metric logging.

The implementation keeps the paper’s layer widths and high-level structure unchanged: hidden dimension $d = 200$, 16 message-passing layers, two attentive readout steps, a 1024-unit dense hidden layer, dropout = 0.1, Adam optimizer, cosine warm restarts, and early stopping with patience 30.

One practical difference from the original repository is that our code unifies pretraining and transfer learning under the same training interface. This makes the pipeline easier to read and reuse. In the current version, the active code path uses a **single shuffled train/validation/test split** for transfer learning rather than the paper’s repeated cross-validation loop on the transfer datasets. We also did not reproduce the paper’s full RIKEN-PlaSMA identification workflow, so that auxiliary experiment is omitted from the main comparison.

Table 1: Datasets used in the final reported experiments and the split sizes produced by our current loader (80%/10%/10% after shuffling). All RT values are stored in seconds.

Dataset	Total	Train	Valid	Test	Role in Report
SMRT	77,980	62,384	7,798	7,798	Base pretraining dataset
Cao_HILIC_116	116	92	12	12	Downstream MiniDataset evaluation
Eawag_XBridgeC18_364	364	291	36	37	Downstream MiniDataset evaluation
FEM_lipids_72	72	57	7	8	Downstream MiniDataset evaluation
FEM_long_412	412	329	41	42	Downstream MiniDataset evaluation
FEM_short_73	73	58	7	8	Downstream MiniDataset evaluation
IPB_Halle_82	82	65	8	9	Downstream MiniDataset evaluation
LIFE_new_184	184	147	18	19	Downstream MiniDataset evaluation
LIFE_old_194	194	155	19	20	Downstream MiniDataset evaluation
MTBLS87_147	147	117	15	15	Downstream MiniDataset evaluation
UniToyama_Atlantis_143	143	114	14	15	Downstream MiniDataset evaluation

Table 2: Training settings used for the runs reported in this report. The MiniDataset stage uses direct SMRT-initialized transfer learning under a single shuffled split.

Stage	Target dataset(s)	Initialization	Batch size	Epoch cap	Notes
Stage 1	SMRT	Random initialization	64	200	Base model training
Stage 2	Each MiniDataset	Best SMRT checkpoint	8	200	Direct transfer with one shuffled split

4 Experimental Setup

4.1 Datasets and Preprocessing

We report only the datasets used in the final reproduction comparison. Table 1 summarizes their sizes and the split sizes actually used by our current code.

The preprocessing pipeline is identical across datasets:

1. remove empty export columns such as `Unnamed:*`;
2. drop rows with missing RT or unparsable molecular structure;
3. prefer SMILES, fall back to InChI when needed;
4. canonicalize molecules with RDKit;
5. convert RT to seconds;
6. transform molecules into PyG graphs and cache them to disk.

One dataset-specific adjustment is important: in the MiniDatasets collection, the raw Excel files store RT in **minutes**, so our preprocessing multiplies those values by 60.0 before training. SMRT is already kept on the paper’s second-based scale.

4.2 Training Configuration

Table 2 lists the hyperparameters used in the code that generated the results currently stored in `Train/Results/DeepGCNRT/`.

The following hyperparameters are shared across the reported stages unless otherwise noted: 16 GCN layers, hidden dimension 200, two attentive readout steps, prediction head $200 \rightarrow 1024 \rightarrow 1$, dropout 0.1, Adam optimizer, `CosineAnnealingWarmRestarts` with $T_0 = 200$, `SmoothL1Loss`, early stopping patience 30, and random seed 42.

Table 3: Explicit deviations from the original paper and why they matter.

Aspect	Paper / official protocol	Our current implementation	Expected impact
SMRT split	Fixed official split: 70,182 train and 7,798 test; train further split 9:1 for validation	<code>SMRT_train_set.txt</code> and <code>SMRT_test_set.txt</code> are merged, then reshuffled into 62,384/7,798/7,798	Makes SMRT results <i>not strictly directly comparable</i> to the paper’s reported test score
Transfer evaluation	10 rounds of 10-fold cross-validation, averaged over seeds; batch size 8	One shuffled train/validation/test split per MiniDataset; batch size 8	Simpler experimental flow, but no longer directly matches the paper’s repeated-CV evaluation protocol
Checkpoint selection	Official transfer script evaluates the held-out fold every epoch and uses that same fold for early stopping and best-checkpoint selection	Our code selects the best checkpoint using a separate validation split and reports the test set only after model selection	Our protocol is methodologically cleaner, while the official protocol can produce optimistically biased transfer scores
Effective training fraction	Each transfer-learning fold trains on roughly 90% of the dataset and does not reserve a separate validation split	Our transfer-learning runs train on 80% of the dataset and reserve 10% for validation	On small datasets, the extra 10% of training data in the official protocol can noticeably improve the reported score
Framework and readout backend	Official implementation uses DGL and its native <code>AttentiveFPReadout</code>	Our reimplementation uses PyG and approximates the readout with graph attention plus <code>GRUCell</code>	Backend-level operator differences can change numerical behavior even when the architecture is conceptually matched
Hyperparameter adaptation	Official results are reported under the released transfer-learning workflow and repeated-CV setting	We intentionally use one fixed hyperparameter set across all MiniDatasets and do not tune settings per dataset	This makes the comparison cleaner, but it does not maximize performance on each individual transfer dataset
Transfer dataset scope	Paper discusses 7 external RPLC datasets	We also ran <code>Cao_HILIC_116</code> , <code>FEM_short_73</code> , and <code>MT-BLS87_147</code> because they appear in the released code-base	Extra coverage, but those rows are supplementary rather than main-paper reproduction
Repetition strategy	3 runs on SMRT; 10×10 CV on transfer learning	One run per stage in the final report	Easier to manage, but variance estimation is weaker than in the paper

4.3 Deviations from the Original Experimental Protocol

Reproduction reports are only useful when deviations are stated clearly. The most important differences between our current implementation and the paper are listed in Table 3.

5 Results and Comparison

5.1 SMRT Benchmark

Our reproduced SMRT base model reaches a test MAE of **26.65 s**, a MedAE of **12.53 s**, a MAPE of **0.0332**, and an R^2 of **0.90**. Numerically, this is very close to the paper’s reported DeepGCN-RT result of 26.55 s MAE. The comparison should still be interpreted with care because our current SMRT loader reshuffles the merged official train and test files before creating the final 80/10/10 split, so the held-out set is not identical to the paper’s fixed benchmark split.

This is the strongest part of the reproduction. Even with the split mismatch, the base model behaves as expected: the large SMRT dataset supports stable optimization, the 16-layer residual graph stack does not collapse, and the edge-aware attentive readout transfers well enough to

Table 4: SMRT test-set performance. Baseline values are reproduced from Table 1 of the paper [1].

Method	MAE (s)	MedAE (s)	MAPE	R^2
DeepGCN-RT (paper)	26.55	12.38	0.033	0.89
DeepGCN-RT (ours)	26.65	12.53	0.033	0.90
GCN (paper)	29.40	15.02	0.040	0.89
DNNpwa (paper)	39.62	25.08	0.050	0.85
GNN-RT (paper)	39.87	25.24	0.050	0.85

Table 5: Transfer-learning MAE comparison on the seven datasets explicitly discussed in the paper. Official numbers are taken from Table 4 of the paper; our runs use a single shuffled train/validation/test split rather than repeated cross-validation.

Dataset	Official MAE (s)	Our MAE (s)	Notes	Gap (s)
Eawag_XBridgeC18_364	45.97	61.78	test $n = 37$	+15.81
FEM_lipids_72	74.48	96.81	test $n = 8$	+22.33
FEM_long_412	110.89	203.27	test $n = 42$	+92.38
IPB_Halle_82	21.20	31.62	test $n = 9$	+10.42
LIFE_new_184	18.13	21.16	test $n = 19$	+3.03
LIFE_old_194	12.07	16.14	test $n = 20$	+4.07
UniToyama_Atlantis_143	50.03	42.94	test $n = 15$	-7.09

serve as a reasonable source model for the later stages.

5.2 MiniDataset Transfer Learning on the Seven Paper Datasets

Table 5 compares our single-split transfer-learning runs against the seven datasets explicitly discussed in the paper. In this group, **UniToyama_Atlantis_143** is the only dataset where our MAE is better than the official DeepGCN-RT number. **LIFE_new_184** and **LIFE_old_194** remain relatively close to the paper. The largest deterioration appears on **FEM_long_412**, where our MAE is worse by 92.38 s.

Across the seven paper datasets, the official DeepGCN-RT MAEs average 47.54 s, whereas our single-split runs average 67.67 s, for a mean gap of +20.14 s. This difference is large enough that it cannot be explained by implementation error alone. The most important reason is the evaluation protocol: the released transfer-learning script performs 10-fold cross-validation but also monitors the held-out fold during training and uses that same fold for early stopping and checkpoint selection. Our pipeline instead uses a separate validation split and reports the test set only after model selection. That choice is more rigorous, but it usually produces less optimistic numbers.

5.3 Additional Datasets Present in the Official Code

The official repository also includes three additional datasets outside the main seven-dataset table. We ran those datasets as supplementary experiments under the same single-split protocol. These extra datasets reinforce the same pattern as the main table: performance is feasible, but highly unstable once the transfer problem becomes data-poor and the evaluation is reduced to a single split.

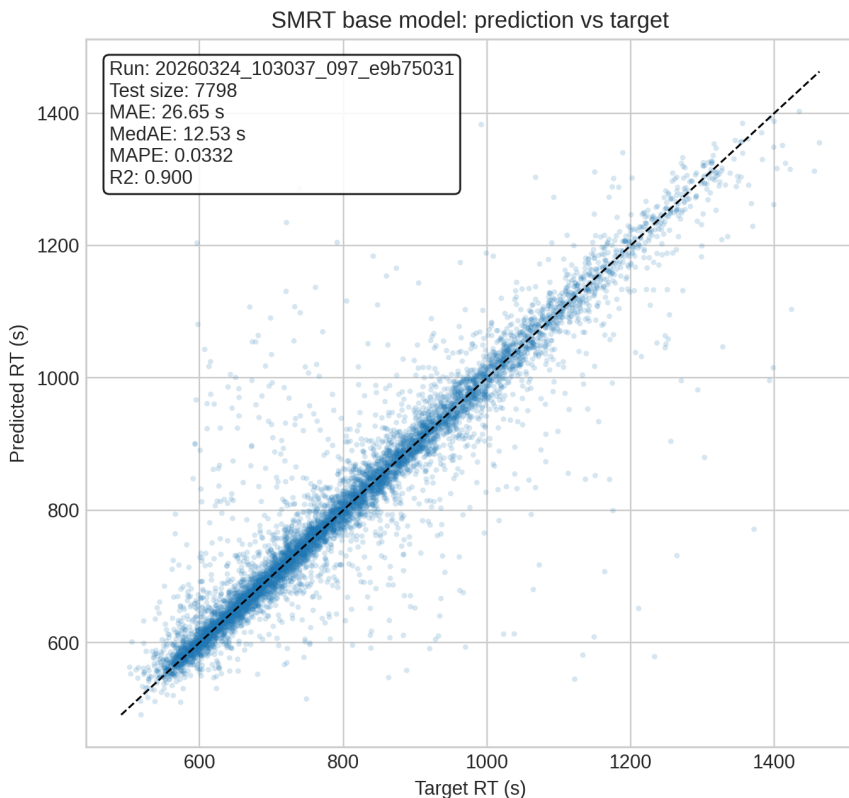


Figure 1: Prediction scatter of the reproduced SMRT base model on the held-out test split. The concentration of points around the diagonal reflects the near-paper-level MAE obtained by our implementation.

Table 6: Supplementary comparison on extra datasets included in the released codebase.

Dataset	Official MAE (s)	Our MAE (s)	Notes	Gap (s)
Cao_HILIC_116	67.64	96.21	test $n = 12$	+28.57
FEM_short_73	77.70	129.60	test $n = 8$	+51.90
MTBLS87_147	79.74	145.60	test $n = 15$	+65.86

6 Analysis and Discussion

6.1 Why Do Some Results Match Well While Others Do Not?

The easiest result to match is the SMRT base model. SMRT is large enough that the optimization dynamics are smooth, the best checkpoint is well defined, and the architecture can show its intended strength: depth plus residual connections without obvious over-smoothing. That is why our reimplementations get close to the paper even though the split itself is not identical. The downstream MiniDataset results are much harder to match. The paper reports repeated 10-fold cross-validation, whereas our runs use one shuffled train/validation/test split. More importantly, the released `transfer_learning.py` script evaluates the held-out fold every epoch and uses that same fold for checkpoint selection and early stopping. This means the official fold score is not a fully untouched test score. Our implementation deliberately avoids that practice and uses a dedicated validation split instead. Methodologically, our setting is cleaner; numerically, it is also more conservative.

The amount of training data also differs. In the official 10-fold setup, each fold trains on roughly 90% of the molecules. In our 80/10/10 split, only 80% of the data is used for fitting because 10%

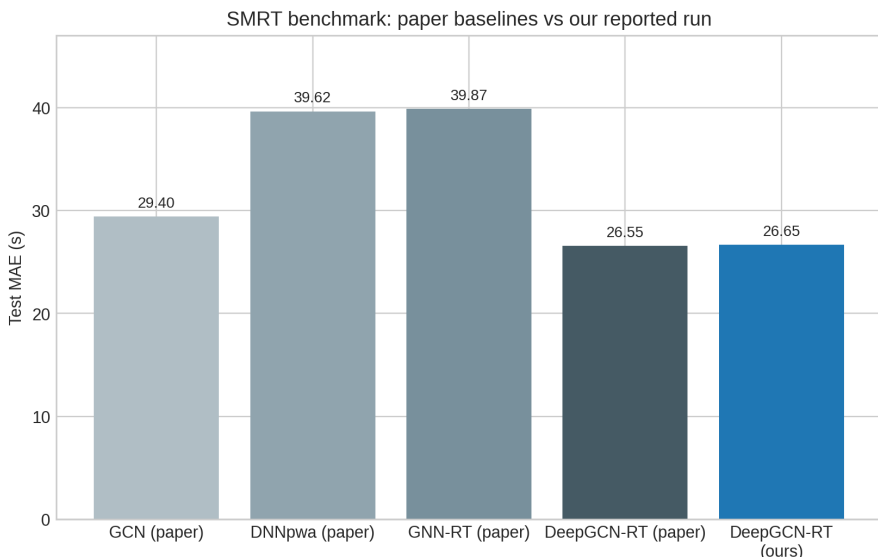


Figure 2: SMRT benchmark MAE comparison between the paper’s published baselines and our reproduced DeepGCN-RT run.

is reserved for validation and 10% for test. On datasets with only 72, 82, or 116 compounds, that 10% reduction in training data is non-trivial. Combined with the stricter model-selection protocol, it is enough to explain why our MiniDataset scores are usually slightly worse than the paper’s.

There is also a framework-level source of mismatch. The official model is implemented in DGL and uses its native `AttentiveFPReadout`. Our model is implemented in PyG, and the readout is approximated with `GATConv` and `GRUCell` because PyG does not provide a drop-in equivalent with the same internals. This is a reasonable reimplementaion choice, but it is still an approximation. Backend-level numerical differences in message passing, batching, and readout can accumulate and cause small performance shifts even when the architecture is conceptually aligned.

6.2 Sensitivity to Hyperparameters and Randomness

The reported transfer-learning results are clearly sensitive to split composition and early stopping. The best epoch ranges from **3** on `Cao_HILIC_116` to **89** on `FEM_long_412`, which already suggests that the smaller datasets do not provide a uniformly stable validation signal. When the test set contains only 8 to 12 compounds, one or two difficult molecules can noticeably change the final MAE.

Because we intentionally fixed the hyperparameters instead of tuning them per dataset, the current report does not claim that these are the best achievable MiniDataset numbers for our implementation. Rather, they are the outcomes of a clean, consistent transfer-learning protocol. A more aggressive optimization campaign could tune learning rate, weight decay, scheduler behavior, and early-stopping patience separately for each MiniDataset and might close part of the remaining gap.

6.3 Strengths and Weaknesses of the Architecture

The reimplementaion still supports the paper’s main architectural claims. First, a 16-layer graph model can be trained effectively for RT prediction when residual connections and a stable readout are present; our SMRT result is direct evidence of that. Second, the model transfers non-trivially across datasets: UniToiyama plus the two LIFE datasets remain competitive even

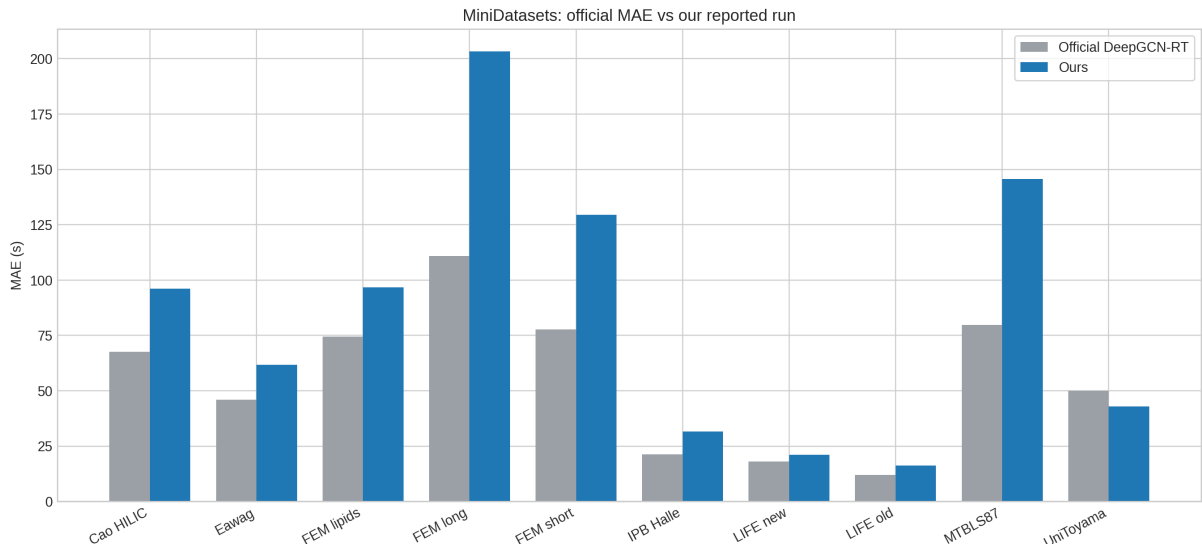


Figure 3: MiniDataset MAE comparison between the official DeepGCN-RT results and our single-split transfer-learning runs.

under a stricter single-split evaluation. Third, the architecture is expressive enough to preserve useful information from both atom and bond features, which is exactly where an edge-aware GNN should help relative to simpler baselines.

The weaknesses are mostly practical rather than conceptual. DeepGCN-RT is still data-hungry on small transfer sets, and its downstream quality depends strongly on evaluation protocol. A separate validation split is cleaner, but it further reduces the already limited training data. In addition, exact framework-level equivalence is difficult when the released implementation depends on DGL-specific components such as `AttentiveFPReadout`.

6.4 What We Learned from the Reimplementation

The reimplementation made two points especially clear. The first is that the model architecture itself is reproducible: once the missing details from the paper are fixed explicitly, a clean PyG implementation can recover near-paper performance on SMRT. The second is that experimental protocol matters almost as much as architecture. Choices about split generation, validation usage, checkpoint selection, and backend implementation can easily dominate the final comparison on small transfer datasets.

7 Conclusion and Lessons Learned

Our independent PyTorch Geometric reimplementation of DeepGCN-RT successfully reproduces the core model behavior. On the SMRT benchmark, the reproduced base model reaches **26.65 s** MAE, which is very close to the paper’s published **26.55 s**.

The main reproduction gap appears on the MiniDatasets. Under our single shuffled split, the transfer-learning results are generally worse than the paper’s repeated cross-validation results, with UniToyama as the main exception and LIFE_new/LIFE_old remaining comparatively close. The most defensible interpretation is therefore not that the architecture failed, but that the downstream evaluation is highly protocol-sensitive once the datasets become small. The official transfer-learning workflow uses the held-out fold during model selection and also keeps more molecules in the training portion of each fold, so its reported numbers are expected to look better than a stricter train/validation/test protocol.

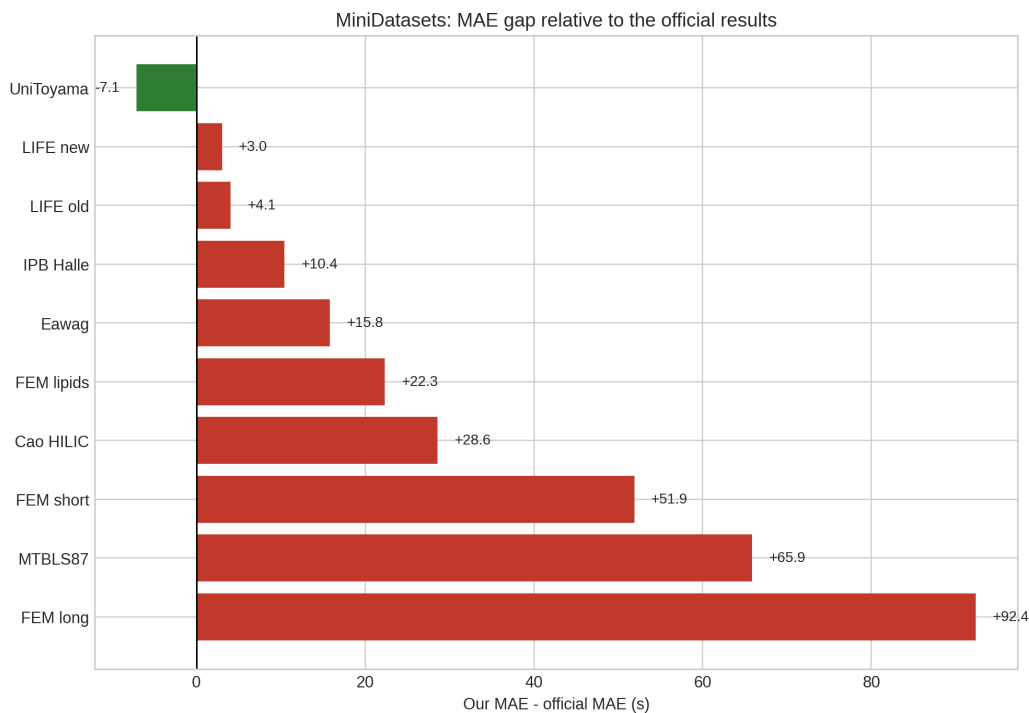


Figure 4: MiniDataset MAE gaps relative to the official DeepGCN-RT numbers. Positive values indicate that our single-split run is worse than the official result.

The main lesson from this project is that reproducing a paper requires reproducing both the model and the measurement procedure. For DeepGCN-RT, the architectural reimplementaion is not the difficult part; the harder issue is aligning data splits, checkpoint selection, readout implementation details, and downstream evaluation with enough fidelity that the final numbers can be compared fairly.

References

- [1] Q. Kang, P. Fang, S. Zhang, H. Qiu, and Z. Lan, “Deep graph convolutional network for small-molecule retention time prediction,” *Journal of Chromatography A*, vol. 1711, 464439, 2023.
- [2] Q. Kang et al., “DeepGCN-RT official source code and pretrained models,” GitHub repository: <https://github.com/kangqiye/DeepGCN-RT>.

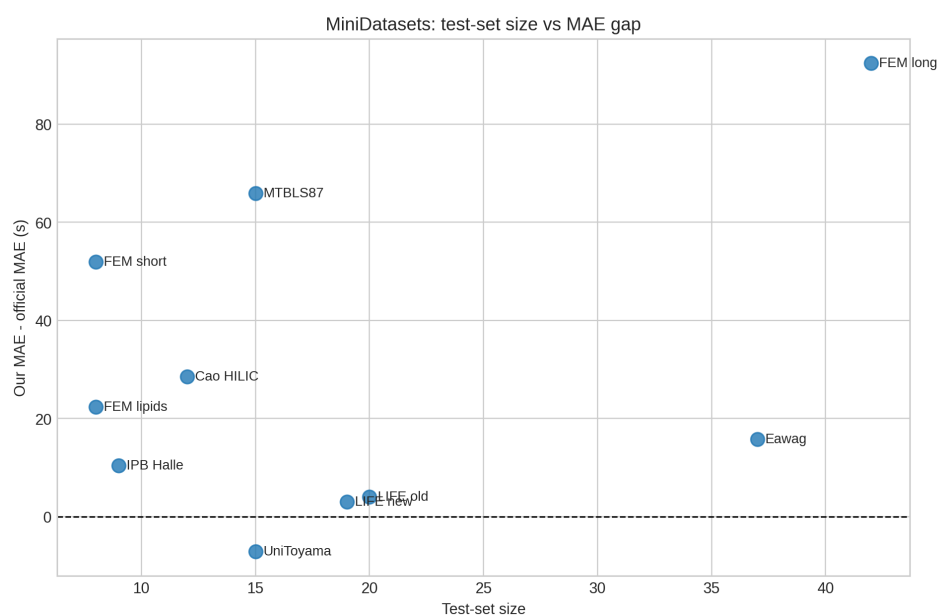


Figure 5: MiniDataset test-set size versus MAE gap. The figure highlights how volatile the comparison becomes once the held-out split contains only 8 to 20 molecules.